Onion AI

Quality Assessment & Grading System

Complete Technical Documentation

Version 1.0.0 | September 2026 | Tharun Balaji

github.com/Tharun1305/onion-final

MODEL	METRIC	SCORE
EfficientNetV2-S	Test Accuracy	94.74%
EfficientNetV2-S	Macro F1-Score	94.03%
YOLOv2-6s	mAP@50	87.74%
YOLOv2-6s	Precision	91.74%

1. System Overview

Onion AI is an end-to-end AI-powered onion quality assessment and grading system designed for APMC (Agricultural Produce Market Committee) procurement centers. It enables quality inspectors to photograph onions and receive instant AI-driven health predictions, replacing manual subjective inspection with objective AI assessment.

Problem: Manual inspection is slow, inconsistent, and cannot reliably detect early-stage diseases like Black Rot or Soft Rot. Inspectors handle thousands of onions per day.

Solution: A two-model AI pipeline (YOLO detector + EfficientNetV2-S classifier) accessible via a cross-platform Flutter app, backed by a FastAPI server.

2. Technology Stack

Layer	Technology	Details
Mobile/Web App	Flutter (Dart)	Android, iOS, Web, Windows
AI Backend	FastAPI (Python)	REST API, /docs Swagger UI
Object Detection	YOLOv2-6s	Onion localization, 640×640
Classification	EfficientNetV2-S (timm)	6-class defect recognition, 384×384
Deep Learning	PyTorch	GPU/CPU inference
Image Processing	Pillow + pillow-heif	JPG, PNG, HEIC/HEIF support
Database	SQLite	Sessions + inspections
State Management	Flutter Provider	Reactive UI state
Authentication	bcrypt + session tokens	30-day secure sessions

3. AI Model Details & Accuracy

3.1 EfficientNetV2-S — Disease Classifier

Architecture: tf_efficientnetv2_s.in21k_ft_in1k (pretrained on ImageNet-21k, fine-tuned)

Input: 384×384 px | **Classes:** 6 | **Epochs:** Up to 40 (early stop patience=8)

Optimizer: AdamW (lr=2e-4) | **Scheduler:** CosineAnnealingLR | **Loss:** CrossEntropyLoss + label smoothing 0.05

■ Overall Test Accuracy: 94.74% (54 / 57 correct)

■ Macro F1-Score: 94.03%

Class	Precision	Recall	F1-Score	Test Samples
black_rot	100.00%	100.00%	100.00%	3
damaged	93.75%	100.00%	96.77%	15

Class	Precision	Recall	F1-Score	Test Samples
healthy	100.00%	86.67%	92.86%	15
mold	100.00%	93.33%	96.55%	15
soft_rot	85.71%	100.00%	92.31%	6
sprouted	75.00%	100.00%	85.71%	3

3.2 Confusion Matrix

Actual \ Predicted	black_rot	damaged	healthy	mold	soft_rot	sprouted
black_rot	3	0	0	0	0	0
damaged	0	15	0	0	0	0
healthy	0	0	13	0	1	1
mold	0	1	0	14	0	0
soft_rot	0	0	0	0	6	0
sprouted	0	0	0	0	0	3

Only 3 misclassifications out of 57 test samples. 2 healthy predicted as soft_rot/sprouted, 1 mold predicted as damaged.

3.3 YOLOv2-6s — Onion Detector

Task: Object detection — locates the onion in image with bounding box

Input: 640×640 px | **Epochs:** 100 | **Batch:** 16 | **IoU:** 0.7

- Precision: 91.74%
- Recall: 79.03%
- mAP@50: 87.74% (Best epoch 91: 87.53%)
- mAP@50-95: 68.89%

4. Complete Workflow: Input → Output

Step 1 — Authentication

Inspector opens app → Login Screen → enters username + password → POST /auth/login → bcrypt verification → session token returned (30-day) → Dashboard

Step 2 — Create Inspection

Inspector enters: Inspector Name, Lot Number, Farmer/Supplier, Location → Local inspection record created in SQLite

Step 3 — Image Capture

Camera/Gallery on device → JPG/PNG/HEIC/WebP supported → Auto EXIF orientation correction → Image stored as bytes in memory

Step 4 — AI Analysis Screen

5-stage progress display: (0) Loading image → (1) Preprocessing → (2) Running AI model → (3) Evaluating probabilities → (4) Finalizing

Step 5 — FastAPI Receives Image

POST /predict (multipart) → Pillow decode → EXIF transpose → RGB convert → Temp .jpg file → calls predict_image()

Step 6 — YOLO Detection

YOLO model runs on 640×640 image → detects onion bounding box (conf ≥ 0.35) → best box selected → image cropped to box (fallback: full image)

Step 7 — EfficientNet Classification

Crop resized to 384×384 → normalized (ImageNet mean/std) → forward pass → 6 logits → softmax → argmax → class + confidence

Step 8 — JSON Response

Server returns:
{"prediction":"black_rot","confidence":90.19,"probabilities":{...},"box":[489,289,611,503],"detected_onion":true}

Step 9 — Quality Grading

OnionQualityGrader.evaluate() → deterministic rules on probabilities → Grade A (healthy ≥70%) / B (50-70%) / C (moderate) / D (rot/severe)

Step 10 — Result Display

AI result screen shows: image + YOLO box, class + confidence, Grade A/B/C/D (color-coded), probability bars, inspector validation option

Step 11 — Save & Sync

POST /inspections/sync → server SQLite → local sqflite DB → offline history → reports generation

5. All Features

5.1 Authentication

Feature	Detail
Fixed accounts	5 pre-configured accounts (1 Admin + 4 Inspectors)
Password security	bcrypt (cost 12), never stored in plaintext
Session length	30-day tokens, server-side SQLite storage
Anti-timing attack	Constant-time dummy check on unknown users
Configuration	Override accounts via .env environment file

5.2 Disease Classes Detected

Class	Display Name	Description
healthy	Healthy	No disease or damage present
damaged	Damaged	Mechanical cuts, bruises, physical damage
black_rot	Black Rot	Fungal disease (Aspergillus niger)
mold	Mold	Surface mold contamination
soft_rot	Soft Rot	Bacterial rot (Erwinia sp.)
sprouted	Sprouted	Green shoot emergence

5.3 Quality Grades

Grade	Label	Criteria
A	Best Quality (Green)	Healthy ≥70%, all rot probabilities <25%
B	Good Quality (Teal)	Healthy 50-70% OR minor sprouting (rot <25%)
C	Lower Quality (Amber)	Damaged / sprouted / emerging rot 25-50%
D	Worst Quality (Red)	Active black rot / soft rot / mold, OR total rot ≥50%

5.4 Platform Support

Platform	Camera	Gallery	AI Analysis	Auth
Android	■	■	■	■
iOS	■	■	■	■
Chrome (Web)	■	■	■	■
Windows Desktop	■	■	■	■

6. API Endpoints Reference

Base URL: http://127.0.0.1:8000

Method	Endpoint	Auth	Description
GET	/	No	Health check
POST	/predict	No	Run AI inference on image
POST	/auth/login	No	Login, get session token
POST	/auth/logout	Bearer	Logout, invalidate token
GET	/auth/me	Bearer	Get current user info
POST	/inspections/sync	Bearer	Save inspection to server
GET	/inspections	Bearer	List user's inspections
POST	/reports/sync	Bearer	Sync report to server
GET	/sync/status	No	Server online timestamp

6.1 /predict Response Example

```
{
  "success": true,
  "prediction": "black_rot",
  "prediction_display": "Black Rot",
  "confidence": 90.19,
  "status": "Defective",
  "probabilities": {
    "black_rot": 90.19, "damaged": 4.21, "healthy": 2.11,
    "mold": 1.89, "soft_rot": 1.05, "sprouted": 0.55
  },
  "detected_onion": true,
  "detection_confidence": 55.01,
  "box": [489, 289, 611, 503]
}
```

7. Deployment & Setup

7.1 Server Setup

```
# In onion-ai/ folder:
python -m venv venv
venv\Scripts\pip install -r requirements.txt

# Start server:
Double-click: start_server.bat
# OR:
venv\Scripts\uvicorn.exe api:app --host 0.0.0.0 --port 8000
# Wait for: Application startup complete
```

7.2 Required Files

File / Folder	Purpose
weights/best.pt	YOLO detector weights (20MB)
models/efficientnetv2_s_multiclass_best.pth	EfficientNet classifier weights (82MB)
api.py	FastAPI server — handles HTTP requests
predict.py	AI inference pipeline (YOLO + EfficientNet)
auth_service.py	Authentication + session management
users.db	SQLite database for sessions
start_server.bat	One-click server startup script

8. Model Accuracy Summary

Model	Metric	Score
EfficientNetV2-S	Test Accuracy	94.74% ■
EfficientNetV2-S	Macro F1-Score	94.03% ■
EfficientNetV2-S	black_rot F1	100.00% ■
EfficientNetV2-S	damaged F1	96.77% ■
EfficientNetV2-S	healthy F1	92.86% ■
EfficientNetV2-S	mold F1	96.55% ■
EfficientNetV2-S	soft_rot F1	92.31% ■
EfficientNetV2-S	sprouted F1	85.71% ■
YOLOv2-6s	mAP@50	87.74% ■
YOLOv2-6s	Precision	91.74% ■
YOLOv2-6s	Recall	79.03% ■
YOLOv2-6s	mAP@50-95	68.89% ■
Combined Pipeline	Inference Speed	0.5–1.2s (CPU)
Combined Pipeline	Classes	6